

Day 28: テストを書こう

今日のゴール

Vitestを使ってテストを書く方法を学びます。tRPCルーターとReactコンポーネントのテストを実装します。

【スクリーンショット: Vitestの実行結果】

なぜこれを学ぶのか？

テストはコードの品質を保証する仕組みです。**テストは車の安全点検のようなもの**。点検をせずに走り続けると、いつか事故を起こします。同様に、テストなしで開発を続けると、いつかバグで大きな問題が起きます。

学習ステップ一覧

ステップ	学習内容	所要時間
Step 1	Vitestのセットアップ確認	10分
Step 2	tRPCルーターのテスト	20分
Step 3	Reactコンポーネントのテスト	20分
Step 4	テストの実行と確認	10分

合計時間: 約60分

Step 1: Vitestのセットアップ確認（10分）

既にインストール済み:

```
# filepath: ターミナル（確認のみ）
$ npm test
# Vitestが起動する
```

設定ファイル確認:

```
// filepath: vitest.config.ts
import { defineConfig } from 'vitest/config';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom',
    globals: true,
  },
});
```

確認ポイント: npm test でVitestが起動する

Step 2: tRPCルーターのテスト（20分）

タスクAPIのテスト:

```
// filepath: src/server/api/routers/__tests__/task.test.t
import { describe, it, expect, beforeEach } from 'vitest'
import { appRouter } from '@server/api/root';
import { db } from '@server/db';

describe('Task Router', () => {
  let caller: ReturnType<typeof appRouter.createCaller>;
  let testUser: { id: string; email: string };
  let testProject: { id: string };

  beforeEach(async () => {
    // テストユーザーを作成
    testUser = await db.user.create({
      data: {
        email: 'test@example.com',
        name: 'Test User',
        password: 'hashedpassword',
      },
    });

    // テストプロジェクトを作成
    testProject = await db.project.create({
      data: {
        name: 'Test Project',
        members: {
          create: {
            userId: testUser.id,
            role: 'OWNER',
          },
        },
      },
    });

    // tRPC caller を作成
    caller = appRouter.createCaller({
      session: {
```

```

        user: testUser,
        expires: '2099-12-31',
      },
      db,
    });
  });

it('タスクを作成できる', async () => {
  const result = await caller.task.create({
    projectId: testProject.id,
    title: 'テストタスク',
    priority: 'HIGH',
  });

  expect(result.title).toBe('テストタスク');
  expect(result.priority).toBe('HIGH');
  expect(result.projectId).toBe(testProject.id);
});

it('タスクを取得できる', async () => {
  // まずタスクを作成
  const created = await caller.task.create({
    projectId: testProject.id,
    title: 'テストタスク',
    priority: 'MEDIUM',
  });

  // 取得
  const result = await caller.task.getById({ id: created.id });

  expect(result?.id).toBe(created.id);
  expect(result?.title).toBe('テストタスク');
});
});

```

確認ポイント: テストが成功する

Step 3: Reactコンポーネントのテスト (20分)

Cardコンポーネントのテスト:

```
// filepath: src/components/dashboard/__tests__/StatsCard
import { describe, it, expect } from 'vitest';
import { render, screen } from '@testing-library/react';
import { StatsCard } from '../StatsCard';
import AssignmentIcon from '@mui/icons-material/Assignment';

describe('StatsCard', () => {
  it('タイトルと値が表示される', () => {
    render(
      <StatsCard
        title="総タスク数"
        value={42}
        icon={<AssignmentIcon />}
        color="#1976d2"
      />
    );

    expect(screen.getByText('総タスク数')).toBeInTheDocument();
    expect(screen.getByText('42')).toBeInTheDocument();
  });

  it('文字列の値も表示できる', () => {
    render(
      <StatsCard
        title="完了率"
        value="75%"
        icon={<AssignmentIcon />}
        color="#2e7d32"
      />
    );

    expect(screen.getByText('完了率')).toBeInTheDocument();
    expect(screen.getByText('75%')).toBeInTheDocument();
  });
});
```

ボタンのクリックテスト:


```
// filepath: src/components/task/__tests__/TaskTimer.test
import { describe, it, expect, vi } from 'vitest';
import { render, screen, fireEvent } from '@testing-library';
import { TaskTimer } from '../TaskTimer';

// Mock tRPC
vi.mock('@trpc/react', () => ({
  api: {
    task: {
      toggleTimer: {
        useMutation: () => ({
          mutate: vi.fn(),
        }),
      },
    },
  },
})));

describe('TaskTimer', () => {
  it('タイマーが表示される', () => {
    render(
      <TaskTimer
        taskId="test-id"
        isTimerActive={false}
        timeSpentMinutes={60}
      />
    );

    expect(screen.getByText('1時間0分')).toBeInTheDocument
  });

  it('開始ボタンをクリックできる', () => {
    render(
      <TaskTimer
        taskId="test-id"
        isTimerActive={false}

```

```
        timeSpentMinutes={0}  
      />  
    );  
  
    const button = screen.getByRole('button');  
    fireEvent.click(button);  
  
    // クリックイベントが発火したことを確認  
    expect(button).toBeInTheDocument();  
  });  
});
```

確認ポイント: コンポーネントのテストが成功する

Step 4: テストの実行と確認（10分）

テストの実行:

```
# filepath: ターミナル（コマンドラインで実行）  
# すべてのテストを実行  
$ npm test  
  
# 特定のファイルだけテスト  
$ npm test task.test.ts  
  
# カバレッジを確認  
$ npm test -- --coverage
```

カバレッジの見方:

File	% Stmts	% Branch	% Funcs	% Li
-----	-----	-----	-----	-----
task.ts	85.71	75.00	80.00	85
StatsCard.tsx	100.00	100.00	100.00	100

- **% Stmts**: 文(statement)のカバレッジ
- **% Branch**: 分岐(if/else)のカバレッジ
- **% Funcs**: 関数のカバレッジ
- **% Lines**: 行のカバレッジ

確認ポイント: 全テストがパスする

学んだこと

- **Vitest**: 高速なテストフレームワーク
- **describe/it/expect**: テストの基本構造
- **beforeEach**: 各テスト前の準備処理
- **render/screen**: Reactコンポーネントのテスト
- **fireEvent**: ユーザー操作のシミュレーション
- **vi.mock**: モックの作成

今日のまとめ

- ☐ Vitestのセットアップを確認できた
- ☐ tRPCルーターのテストを書けた
- ☐ Reactコンポーネントのテストを書けた
- ☐ テストを実行して確認できた

⚠ テストのベストプラクティス

原則	説明
1機能1テスト	1つのitで1つの機能をテスト
AAA パターン	Arrange(準備) → Act(実行) → Assert(検証)
テストは独立させる	他のテストに依存しない
意味のある名前	テストが何を確認するか明確に

次回予告

Day 29では、最終調整と動作確認を行います。